Cybersecurity + Malware Analysis + Reverse Engineering + Macos +

macOS Malware Analysis — Part 1: Understanding Mach-O

13 min read · Sep 5, 2026



CHANDRA KANT BAURI Follow

Listen Share

Exploring how macOS binaries are structured, loaded, and executed through a hands-on analysis of a real RustBucket payload.

If you come from Windows malware analysis, your instincts lean toward Portable Executable (PE) headers, DOS stubs, import address tables, and Registry persistence keys.

When switching to macOS, that background can feel disorienting. The operating system does not use PE files or a centralized Registry. Instead, binaries compile to the Mach-O (Mach Object) format, load through Apple's dynamic linker (**dyld**), and run on Mach kernel primitives layered beneath a BSD/POSIX subsystem.

This guide walks through macOS binary analysis from the ground up. Instead of leaning on abstract documentation, we examine an actual malware sample: the second-stage payload from the **RustBucket campaign**, attributed to the **BlueNorOff group**

Sample details

```
Malware family:     RustBucket (Stage 2 payload)
Dropped file name:  .pd
SHA-256:            7887638bcafd57e2896c7c16698e927ce92fd7d409aae698d33cdca3ce8
```

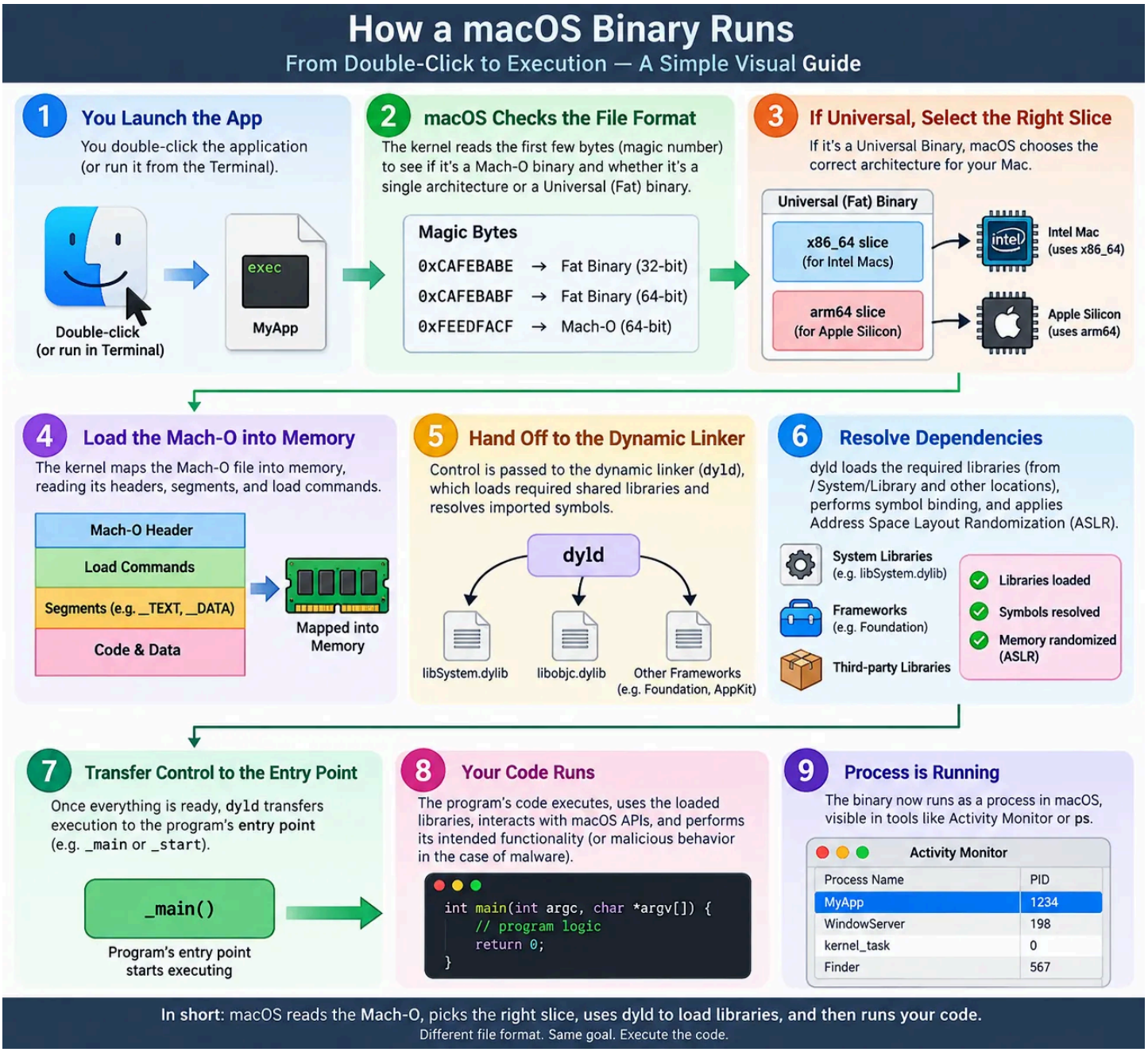
Analysis tools

You do not need paid commercial disassemblers to perform initial triage. The standard Apple Command Line Tools package (**xcode-select — install**) includes the necessary inspection binaries:

file: Identifies file formats and compiled architectures.
otool: Displays specific sections and headers within Mach-O files.
lipo: Extracts or combines architecture slices in universal binaries.
nm: Lists symbol table entries.
strings: Scrapes printable text sequences.
codesign: Extracts and verifies digital signatures and code directory hashes.

Open in app ↗ Sign up Sign in

Medium Search



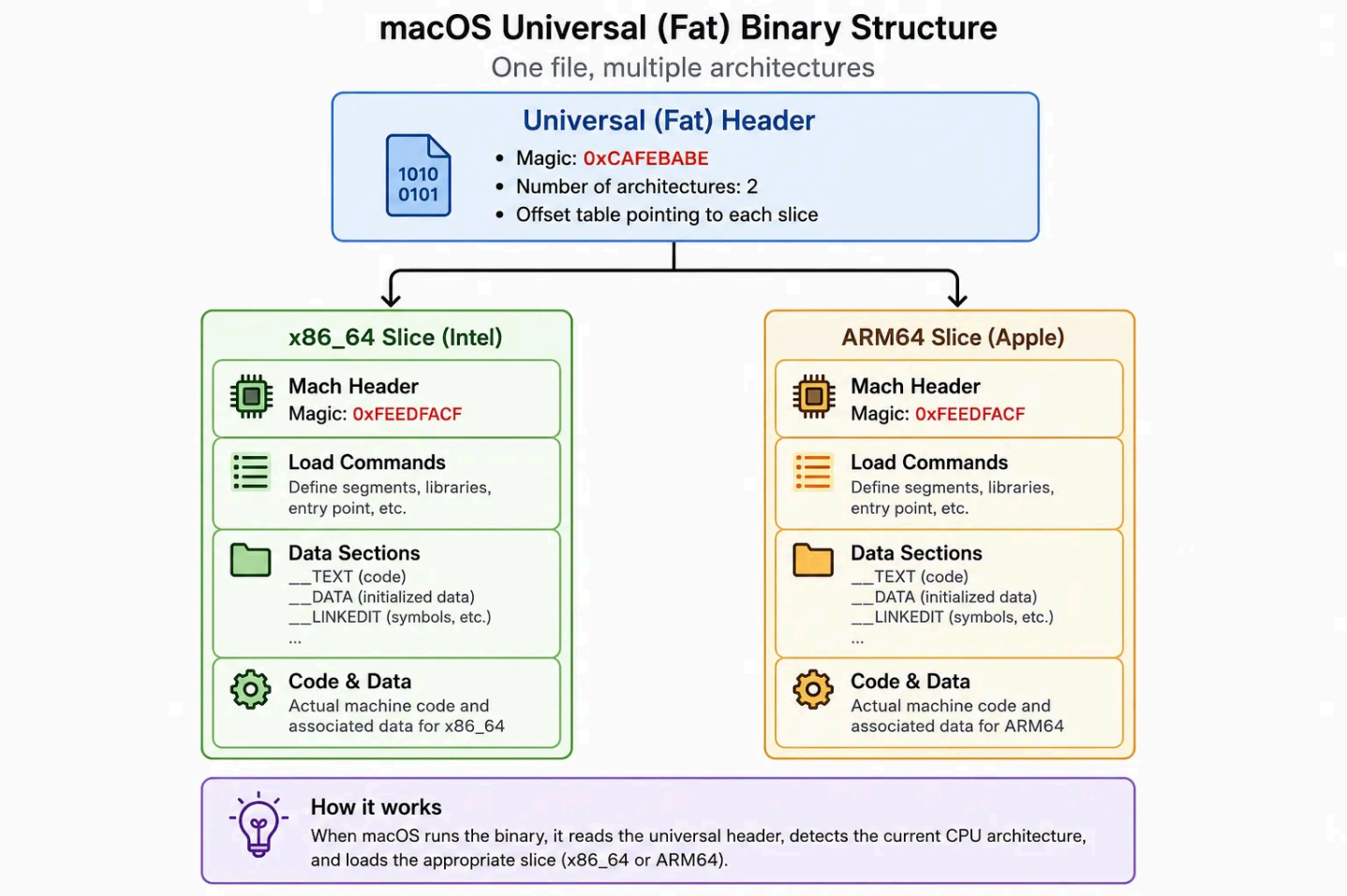
How a macOS binary moves from disk through the kernel and dynamic linker into memory.

Universal binaries and Mach-O structure

On Windows, an executable targets one architecture at a time. A single 64-bit PE file does not simultaneously package x86_64 and ARM64 machine code.

Apple addresses multi-architecture hardware using universal binaries, historically known as fat binaries. A universal binary is an archive wrapper that packages

independent Mach-O slices for different processor architectures into a single container file.



Universal binary layout

A universal binary acts as a container holding separate Mach-O slices for each supported architecture.

When a user executes a universal binary, the dynamic linker inspects the fat header:

- On an Apple Silicon Mac (M1 through M4 chips), the system loads the native **arm64** slice.
- On an Intel Mac (or on Apple Silicon running under Rosetta 2 emulation), the system selects the **x86_64** slice.
- Single-architecture binaries omit the fat container entirely and begin immediately with the native Mach-O header.

Magic bytes reference table

Keep these identification constants in mind when inspecting binary headers in a hex editor or debugger:

Target Type	Magic Bytes (Hex)	Size	Constant
Fat binary (32-bit offset)	0xCAFEBABE	4 bytes	FAT_MAGIC
Fat binary (64-bit offset)	0xCAFEBABF	4 bytes	FAT_MAGIC_64
Mach-O 64-bit (x86_64)	0xFEEDFACF	4 bytes	CPU_TYPE_X86_64
Mach-O 64-bit (ARM64)	0xFEEDFACF	4 bytes	CPU_TYPE_ARM64

> ***Note:*** *Although each magic value appears as 8 characters (for example,* `0xCAFEBABE` *), it is written in* ***hexadecimal notation****, where* ***2 hexadecimal characters represent 1 byte****. Therefore, 8 hexadecimal characters represent* ***4 bytes (32 bits)****.*

Both 64-bit slices share the same Mach-O magic number (**0xFEEDFACF**). The operating system tells them apart using the CPU type value located immediately after the magic bytes.

Inspecting the RustBucket sample

We begin by examining the dropped payload (**.pd**) directly from the terminal.

Step 1: check the file type with file

Run **file** on the sample to view basic header information:

```
file .pd
```

```
kant@APPLEs-MacBook-Pro ~/test> file .pd
.pd: Mach-O universal binary with 2 architectures: [x86_64:\012- Mach-O 64-bit x86_64 executable, flags:<NOUNDEFS|DYLDLINK|TWOLEVEL|PIE>] [\012- arm64:\012- Mach-O 64-bit arm64 executable, flags:<NOUNDEFS|DYLDLINK|TWOLEVEL|PIE>]
```

file command output

The file command identifies both slices and active header flags.

Mach-O universal binary with 2 architectures: The payload packages two compiled binaries within one file. **x86_64** and **arm64: The author compiled native code for both Intel hardware and modern Apple Silicon chips.**
Header flags:

- **NOUNDEFS:** All internal references were resolved at link time.
- **DYLDLINK:** Directs the kernel to hand execution over to **dyld.**
- **TWOLEVEL:** Enables two-level namespace resolution. Every imported symbol records both the symbol name and the specific library exporting it.
- **PIE:** Position Independent Executable. The kernel loads the binary at a randomized virtual memory address on each run (ASLR).

Step 2: read the fat header with otool -f

Because .pd is a universal binary, it starts with a fat header rather than code instructions. This header acts as a table of contents, telling `dyld` the exact byte offset where each architecture slice begins.

Run **otool -f** to display the container layout:

```
otool -f .pd
```

```
kant@APPLEs-MacBook-Pro ~/test> otool -f .pd
Fat headers
fat_magic 0xcafebabe
nfat_arch 2
architecture 0
    cputype 16777223
    cpusubtype 3
    capabilities 0x0
    offset 16384
    size 72480
    align 2^14 (16384)
architecture 1
    cputype 16777228
    cpusubtype 0
    capabilities 0x0
    offset 98304
    size 82944
    align 2^14 (16384)
```

otool -f output

Inspecting the fat header reveals where each slice begins and how much space it occupies.

The output breaks down as follows:

- **fat_magic 0xcafebabe:** Identifies a standard 32-bit universal container.
- **nfat_arch 2:** Specifies that the file contains exactly two architecture slices.

Here is how the two slices compare:

Field	Architecture 0 (x86_64)	Architecture 1 (arm64)	Why It Matters
`cputype`	`16777223` (`0x01000007`)	`16777228` (`0x0100000C`)	Maps to `CPU_TYPE_X86_64` and `CPU_TYPE_ARM64` .
`cpusubtype`	`3` (`CPU_SUBTYPE_I386_ALL`)	`0` (`CPU_SUBTYPE_ARM64_ALL`)	Indicates standard processor capabilities without specialized instructions.
`offset`	`16384` (`0x4000`)	`98304` (`0x18000`)	The exact byte offset in the file where each independent Mach-O binary begins.
`size`	`72480` bytes (~70.8 KB)	`82944` bytes (~81 KB)	ARM64 instructions are fixed-width (4 bytes each), typically making the ARM64 slice slightly larger than the variable-length x86_64 slice.
`align`	`2^14` (16,384 bytes)	`2^14` (16,384 bytes)	Memory page alignment.

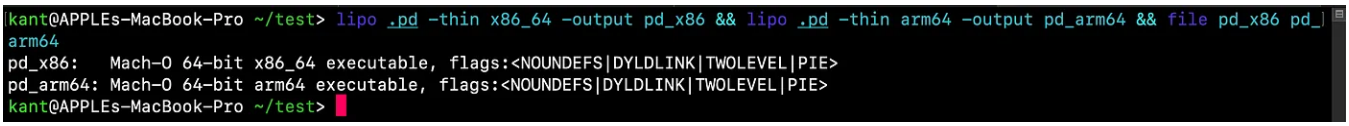
> *Analyst Note on Alignment: Both slices share an alignment value of 2^{14} (16 KB). Intel x86_64 processors traditionally use 4 KB memory pages, whereas Apple Silicon uses 16 KB pages. Setting 16 KB page alignment for both slices ensures that the binary maps into memory on modern macOS systems without paging faults.*

Step 3: split the binary into individual slices with lipo

Analyzing a universal binary directly in tools like Ghidra, Hopper, or IDA Pro requires choosing one architecture at a time. Slicing the file also prevents command-line tools from repeating output for both architectures.

Use **lipo** with the **-thin** flag to carve each architecture into its own file:

```
lipo .pd -thin x86_64 -output pd_x86 && lipo .pd -thin arm64 -output pd_arm64 &
```



Splitting the binary with lipo

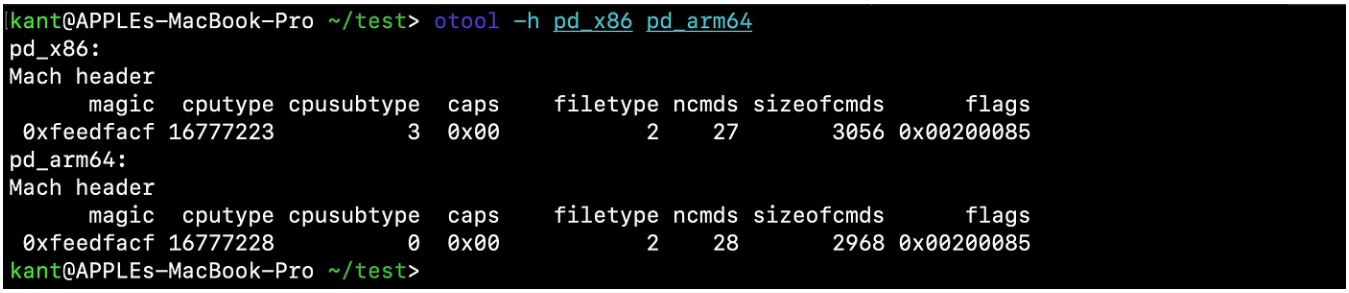
Carving the universal binary yields two standalone 64-bit Mach-O executables.

Thinning the binary strips the initial 16 KB fat container and padding. Both **pd_x86** and **pd_arm64** now start at offset **0x00** with the native 64-bit Mach-O magic number (**0xFEEDFACF**). You can now reverse engineer each architecture independently.

Step 4: inspect native Mach-O headers with otool -h

Each extracted slice begins with a **mach_header_64** structure (defined in <mach-o/loader.h>). We can compare their headers side by side:

```
otool -h pd_x86 pd_arm64
```



otool -h output for both slices

Comparing headers shows differences in load command counts between architectures.

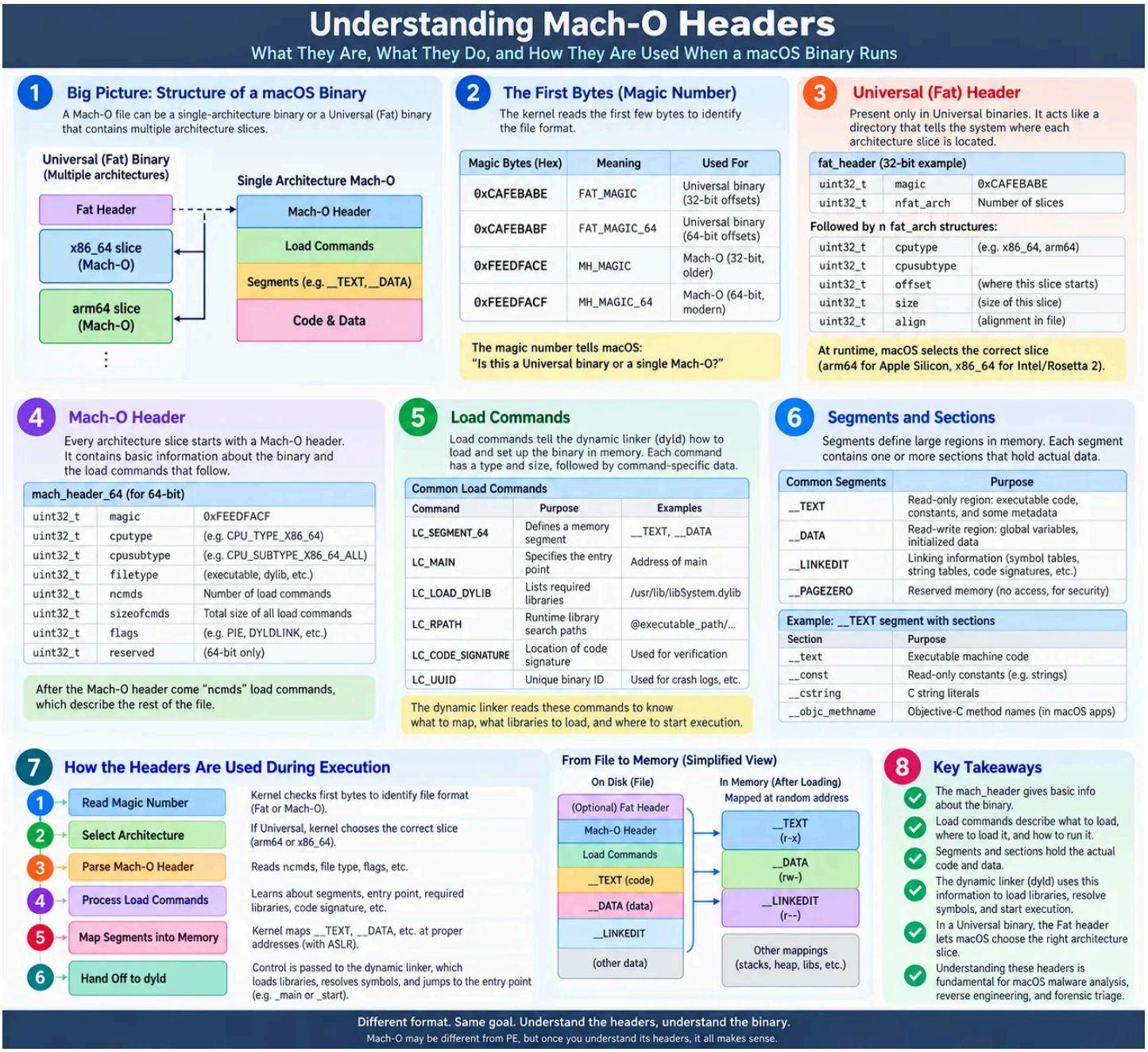
Field	pd_x86 Value	pd_arm64 Value	Meaning & Internals Detail
`magic`	`0xfeedfacf`	`0xfeedfacf`	`MH_MAGIC_64`. Confirms both are 64-bit Mach-O binaries in native host byte order.
`cputype`	`16777223` (`0x01000007`)	`16777228` (`0x0100000C`)	`CPU_TYPE_X86_64` vs `CPU_TYPE_ARM64`.
`cpusubtype`	`3`	`0`	`CPU_SUBTYPE_I386_ALL` vs `CPU_SUBTYPE_ARM64_ALL`.
`caps`	`0x00`	`0x00`	Capability bits. On ARM64e systems, this field can hold pointer authentication (PAC) flags.
`filetype`	`2`	`2`	`MH_EXECUTE`. Tells `dyld` this is an executable binary with an entry point, not a shared library (`MH_DYLIB` = 6) or object bundle (`MH_BUNDLE` = 8).
`ncmds`	`27`	`28`	The total count of Load Commands immediately following the header. Notice ARM64 has 28 vs. 27 on x86_64!
`sizeofcmds`	`3056` bytes	`2968` bytes	The total byte size occupied by all load command structures combined.
`flags`	`0x00200085`	`0x00200085`	Bitmask of runtime capabilities.

Notice that **pd_arm64** contains 28 load commands, while **pd_x86** has 27. We will see why in the next section.

The header fields provide the kernel with basic information about the binary.

The flags field contains the combined bitmask **0x00200085**:

- `0x00000001` (`MH_NOUNDEFS`): The compiler resolved all internal symbols, no undefined references remain.
- `0x00000004` (`MH_DYLDLINK`): Binary uses the dynamic linker (`dyld`) rather than static linking.
- `0x00000080` (`MH_TWOLEVEL`): Uses two-level namespace symbol resolution (each import records both its symbol name and the specific library it belongs to, speeding up loading and preventing namespace collisions).
- `0x00200000` (`MH_PIE`): **Position Independent Executable.** ASLR (Address Space Layout Randomization) will randomize the base virtual address on every run.



Mach-O header structure

Step 5: read the load commands

Load commands follow immediately after the Mach-O header. They act as instructions for the kernel and dyld, defining:

- How segments map into virtual memory.
- The location of the entry point.
- Which shared dynamic libraries (**.dylib**) the process requires.
- Where to find symbol tables, string tables, and digital signatures.

We can list the first 15 load commands of **pd_x86** using **otool -l:**

```
otool -l pd_x86 | grep "cmd " | head -n 15
```

```
kant@APPLEs-MacBook-Pro ~/test> otool -l pd_x86 | grep "cmd " | head -n 15
      cmd LC_SEGMENT_64
      cmd LC_SEGMENT_64
      cmd LC_SEGMENT_64
      cmd LC_SEGMENT_64
            cmd LC_DYLD_INFO_ONLY
     cmd LC_SYMTAB
            cmd LC_DYSYMTAB
         cmd LC_LOAD_DYLINKER
     cmd LC_UUID
      cmd LC_VERSION_MIN_MACOSX
      cmd LC_SOURCE_VERSION
       cmd LC_MAIN
          cmd LC_LOAD_DYLIB
          cmd LC_LOAD_DYLIB
          cmd LC_LOAD_DYLIB
kant@APPLEs-MacBook-Pro ~/test>
```

Load commands for x86_64

The first 15 load commands describe memory segments and linker requirements.

The main commands perform specific functions:

- **`LC_SEGMENT_64` (x4):** Directs the kernel to map four primary memory regions into the virtual address space: `__PAGEZERO`, `__TEXT`, `__DATA`, and `__LINKEDIT`.
- **`LC_DYLD_INFO_ONLY`:** Provides the compressed rebase, bind, weak-bind, lazy-bind, and export opcodes used by `dyld` during ASLR rebasing and symbol resolution.
- **`LC_SYMTAB` & `LC_DYSYMTAB`:** Pointers to the static and dynamic symbol tables (function names, variable names, and relocation entries).
- **`LC_LOAD_DYLINKER`:** Path to the dynamic linker responsible for bootstrapping this process (almost universally `/usr/lib/dyld`).
- **`LC_UUID`:** A unique 128-bit build identifier matched with debug symbols (`.dSYM`).
- **`LC_VERSION_MIN_MACOSX`:** The minimum macOS SDK version required to execute this slice.

- **`LC_MAIN`** : The replacement for the legacy `LC_UNIXTHREAD` command. Specifies the offset to the program's initial entry point (`main`).
- **`LC_LOAD_DYLIB`** : Dynamically linked libraries/frameworks required by the malware at runtime.

Step 6: compare load commands between architectures

Running the same command on the ARM64 slice reveals structural differences:

```
otool -l pd_arm64 | grep "cmd " | head -n 15
```



Load commands for ARM64

The ARM64 slice introduces modern segment and version commands.

Get CHANDRA KANT BAURI's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

Two structural differences explain why the ARM64 slice has 28 commands instead of 27:

Feature	pd_x86 (Intel)	pd_arm64 (Apple Silicon)	What It Means for Malware Analysis
Segment Count	4 segments	**5 segments**	ARM64 splits constant data into its own dedicated segment (`__DATA_CONST`) alongside `__DATA` , strengthening page permission protections.
Version Command	`LC_VERSION_MIN_MACOSX`	**`LC_BUILD_VERSION`**	`LC_VERSION_MIN_MACOSX` is the legacy load command. Apple Silicon uses `LC_BUILD_VERSION` , which specifies the target platform, minimum OS version, and the exact SDK version used to compile the binary.

Step 7: Mach-O segments and section structure

In Mach-O terminology, a segment is a memory container mapped directly by the kernel with a defined set of page permissions (read, write, execute). Inside segments lie individual sections, which hold code instructions, constants, and pointers.

We can list all segments declared in both slices:

```
otool -l pd_x86 | grep -E “(cmd LC_SEGMENT_64|segname)”
otool -l pd_arm64 | grep -E “(cmd LC_SEGMENT_64|segname)”
```

```
kant@APPLEs-MacBook-Pro ~/test> otool -l pd_x86 | grep -E "(cmd LC_SEGMENT_64|segname)"
      cmd LC_SEGMENT_64
  segname __PAGEZERO
      cmd LC_SEGMENT_64
  segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
      cmd LC_SEGMENT_64
  segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
      cmd LC_SEGMENT_64
  segname __LINKEDIT
```

Segment list for x86_64

The Intel slice defines four memory segments.

```
kant@APPLEs-MacBook-Pro ~/test> otool -l pd_arm64 | grep -E "(cmd LC_SEGMENT_64|segname)"
      cmd LC_SEGMENT_64
  segname __PAGEZERO
      cmd LC_SEGMENT_64
  segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
   segname __TEXT
      cmd LC_SEGMENT_64
  segname __DATA_CONST
   segname __DATA_CONST
   segname __DATA_CONST
   segname __DATA_CONST
      cmd LC_SEGMENT_64
  segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
   segname __DATA
      cmd LC_SEGMENT_64
  segname __LINKEDIT
```

Segment list for ARM64

The ARM64 slice adds a fifth segment, __DATA_CONST.

```
[pd_x86]
  cmd LC_SEGMENT_64 -> segname __PAGEZERO
  cmd LC_SEGMENT_64 -> segname __TEXT        (11 sections)
  cmd LC_SEGMENT_64 -> segname __DATA        (12 sections)
  cmd LC_SEGMENT_64 -> segname __LINKEDIT

[pd_arm64]
  cmd LC_SEGMENT_64 -> segname __PAGEZERO
  cmd LC_SEGMENT_64 -> segname __TEXT        (11 sections)
  cmd LC_SEGMENT_64 -> segname __DATA_CONST  (3 sections)
  cmd LC_SEGMENT_64 -> segname __DATA        (5 sections)
  cmd LC_SEGMENT_64 -> segname __LINKEDIT
```

Each segment carries distinct permissions and duties:

Segment	Permissions	Purpose
`__PAGEZERO`	`---` (None)	A non-executable, non-readable guard block starting at virtual address `0x0`. Catches NULL pointer dereferences by immediately triggering an `EXC_BAD_ACCESS` crash if the malware tries to read/write to `0x0`.
`__TEXT`	`r-x` (Read / Exec)	Contains the compiled machine instructions and immutable data (like string literals). It is read-only to prevent self-modifying code.
`__DATA_CONST`	`r--` (Read-only)	*ARM64 / Modern macOS only.* Holds pointers and structures initialized at load time by `dyld` and then immediately locked down to read-only before execution starts.
`__DATA`	`rw-` (Read / Write)	Mutable global variables, static storage, and lazy symbol pointers that are modified at runtime.
`__LINKEDIT`	`r--` (Read-only)	Raw dynamic link metadata, including export tables, symbol tables, code signatures, and relocation info used by `dyld`.

Step 8: virtual memory protection with __PAGEZERO

To see how macOS enforces memory boundaries, inspect the details of the __PAGEZERO segment:

```
otool -l pd_arm64 | grep -A 8 “segname __PAGEZERO”
```

```
kant@APPLEs-MacBook-Pro ~/test> otool -l pd_arm64 | grep -A 8 "segname __PAGEZERO"
  segname __PAGEZERO
   vmaddr 0x0000000000000000
   vmsize 0x0000000100000000
  fileoff 0
 filesize 0
  maxprot 0x00000000
 initprot 0x00000000
   nsects 0
    flags 0x0
```

__PAGEZERO segment output

The __PAGEZERO segment covers the lower 4 GB of virtual address space without consuming disk space.

The fields reveal three points about memory safety:

- **`vmaddr 0x0000000000000000` & `vmsize 0x0000000100000000`:**
  The segment begins at virtual address `0x0` and extends for `0x100000000` bytes (exactly **4 GB** in size). This 4 GB region covers the entire 32-bit lower address range.
- **`fileoff 0` & `filesize 0`:**
  It consumes zero bytes on disk. It is purely a virtual memory allocation made by the kernel at runtime.
- **`initprot 0x0` & `maxprot 0x0`:**
  Initial and maximum page permissions are both `0` (`---`, no read, no write, no execute).

> **Why This Matters for Malware Analysis:**
> If a developer or malware author introduces a NULL pointer dereference (or tries to access memory via a corrupted 32-bit integer offset inside this 4 GB window), the CPU immediately faults with `EXC_BAD_ACCESS (SIGSEGV)` rather than allowing an exploit to hijack execution via NULL page spraying.

Step 9: calculate the entry point in __TEXT

We can locate the entry point using the **__TEXT** segment definition and the **LC_MAIN** load command:

```
otool -l pd_arm64 | grep -A 8 "segname __TEXT" | head -n 9
otool -l pd_arm64 | grep -A 4 "cmd LC_MAIN"
```



__TEXT segment and LC_MAIN output

```
segname __TEXT
   vmaddr 0x0000000100000000
   vmsize 0x0000000000004000
  fileoff 0
 filesize 16384
  maxprot 0x00000005
 initprot 0x00000005
   nsects 11
    flags 0x0
---
       cmd LC_MAIN
   cmdsize 24
  entryoff 9892
 stacksize 0
```

Deconstructing `__TEXT` Memory Setup

- **`vmaddr 0x0000000100000000`** : As expected, `__TEXT` begins immediately where `__PAGEZERO` ends (at the 4 GB mark, `0x100000000`).
- **`initprot 0x00000005`** : In octal/hex permissions, `0x5` corresponds to **Read + Execute** (4 + 1 = 5, `r-x`). It is non-writable.
- **`filesize 16384`** : Exactly 16 KB (aligned to Apple Silicon page size).

Demystifying `LC_MAIN` & The Entry Point

On legacy Mach-O binaries, execution entry was governed by `LC_UNIXTHREAD`, which explicitly supplied CPU register states (`rip` or `pc`). Modern macOS uses `LC_MAIN`:

- **`entryoff 9892`:** This is a raw relative byte offset from the start of the binary image to the first instruction of `main()`.
- **Calculating the Virtual Memory Entry Point:**

```
VM Base + entryoff = 0x100000000 + 9892 (0x26A4) = 0x1000026A4
```

When you open pd_arm64 in Ghidra or IDA Pro, disassembly begins at **0x1000026A4.**

Step 10: map shared dependencies with otool -L

Before running code, **dyld** parses every **LC_LOAD_DYLIB** command to map required frameworks into memory. Checking linked libraries is the macOS counterpart to inspecting the Import Address Table (IAT) on Windows:

```
otool -L pd_arm64
```

```
kant@APPLEs-MacBook-Pro ~/test> otool -L pd_arm64
pd_arm64:
        /System/Library/Frameworks/Foundation.framework/Versions/C/Foundation (compatibility version 300.0.0, current version 1953.255.0)
        /usr/lib/libobjc.A.dylib (compatibility version 1.0.0, current version 228.0.0)
        /usr/lib/libSystem.B.dylib (compatibility version 1.0.0, current version 1319.0.0)
        /usr/lib/swift/libswiftCore.dylib (compatibility version 1.0.0, current version 5.7.1)
        /usr/lib/swift/libswiftCoreFoundation.dylib (compatibility version 1.0.0, current version 120.100.0, weak)
        /usr/lib/swift/libswiftDarwin.dylib (compatibility version 1.0.0, current version 0.0.0)
        /usr/lib/swift/libswiftDispatch.dylib (compatibility version 1.0.0, current version 17.0.0)
        /usr/lib/swift/libswiftIOKit.dylib (compatibility version 1.0.0, current version 1.0.0, weak)
        /usr/lib/swift/libswiftObjectiveC.dylib (compatibility version 1.0.0, current version 6.0.0, weak)
        /usr/lib/swift/libswiftXPC.dylib (compatibility version 1.0.0, current version 6.0.0, weak)
        /usr/lib/swift/libswiftFoundation.dylib (compatibility version 1.0.0, current version 1.0.0)
```

Dynamic libraries listed with otool -L

Linked dynamic libraries reveal runtime dependencies and language choices.

- **Written in Swift:** The prevalence of `/usr/lib/swift/libswift*.dylib` indicates this payload is written in **Swift** rather than pure C or Objective-C. When

reversing in Ghidra or IDA Pro later, function names will follow Swift name mangling conventions (`$s...`).

- **High-Level OS Interactions (`Foundation.framework`):** Foundation provides classes like `FileManager` , `URLSession` , and `Process` . This points to standard file dropping, HTTP network requests, or launching subprocesses.
- **Low-Level Subsystem Access (`libswiftIOKit.dylib` & `libswiftXPC.dylib`):**
- `IOKit` : Often leveraged by malware for hardware profiling, gathering serial numbers, or detecting virtualized/sandbox environments.
- `XPC` : macOS's native Inter-Process Communication (IPC) framework, often used to communicate with system daemons or escalate privileges.
- **`libSystem.B.dylib`** : The core Darwin umbrella library containing libc, libdispatch, POSIX calls, and Mach messaging primitives.

Step 11: inspect string literals from __cstring

Running **strings** over an entire binary often produces noise, including mangled symbols, protocol tables, and linker metadata.

In Mach-O executables, literal ASCII strings placed in code reside inside the **__cstring** section of the **__TEXT** segment. We can dump just this section with virtual memory addresses using **otool:**

```
otool -v -s __TEXT __cstring pd_arm64 | head -n 35
```

```
kant@APPLEs-MacBook-Pro ~/test> otool -v -s __TEXT __cstring pd_arm64 | head -n 35
pd_arm64:
Contents of (__TEXT,__cstring) section
0000000100003d70  mozilla/4.0 (compatible; msie 8.0; windows nt 5.1; trident/4.0)
0000000100003db0  v32@?0@"NSData"8@"NSURLResponse"16@"NSError"24
0000000100003ddf
0000000100003de0  HTTP Request Failed
0000000100003df5  wb
0000000100003df8  v8@?0
0000000100003dfe  swift_getFunctionTypeMetadataGlobalActor
0000000100003e27  swift_getFunctionTypeMetadataGlobalActorStandalone
kant@APPLEs-MacBook-Pro ~/test>
```

Strings in the __cstring section

The __cstring section contains embedded string literals from the source code.

This output exposes operational indicators:

- **Hardcoded User-Agent string:**

```
mozilla/4.0 (compatible; msie 8.0; windows nt 5.1; trident/4.0)
```

An Apple Silicon binary using a Windows XP and Internet Explorer 8 User-Agent string is a clear anomaly. The malware author hardcoded this header to blend into network traffic or evade basic macOS detection rules.

- **Objective-C completion handler signature:**

```
v32@?0@"NSData"8@"NSURLResponse"16@"NSError"24
```

This type encoding corresponds to the block signature (void)(**NSData** , **NSURLResponse** , **NSError**). It confirms that the binary uses **URLSession dataTaskWithRequest:completionHandler:** for command-and-control communication.

- **Disk and error operations:**
- **HTTP Request Failed**: Error message printed after failed network beacons.
- **wb**: Standard POSIX file write mode (write binary), typically used with **fopen()** when writing downloaded payloads to disk.

Step 12: inspect code signing and Gatekeeper attributes

Every executable running on Apple Silicon must have a cryptographic signature. If an ARM64 binary lacks a signature entirely, the Mach kernel kills the process at

launch (**SIGKILL / CODESIGNING**).

We can inspect the signature using **codesign:**

```
codesign -dv — verbose=4 pd_arm64
```

```
kant@APPLEs-MacBook-Pro ~/test> codesign -dv --verbose=4 pd_arm64
Executable=/Users/kant/test/pd_arm64
Identifier=dd-55554944e4546013db0f3585bc706be2455d084e
Format=Mach-O thin (arm64)
CodeDirectory v=20400 size=868 flags=0x2(adhoc) hashes=16+7 location=embedded
VersionPlatform=1
VersionMin=720896
VersionSDK=852224
Hash type=sha256 size=32
CandidateCDHash sha256=90089ad92e2eb0688a8ea060a752e18128539253
CandidateCDHashFull sha256=90089ad92e2eb0688a8ea060a752e18128539253d6a9af1b310c036b89d67010
Hash choices=sha256
CMSDigest=90089ad92e2eb0688a8ea060a752e18128539253d6a9af1b310c036b89d67010
CMSDigestType=2
Executable Segment base=0
Executable Segment limit=16384
Executable Segment flags=0x1
Page size=4096
CDHash=90089ad92e2eb0688a8ea060a752e18128539253
Signature=adhoc
Info.plist=not bound
TeamIdentifier=not set
Sealed Resources=none
Internal requirements count=0 size=12
kant@APPLEs-MacBook-Pro ~/test>
```

Code signing details from codesign

The codesign utility displays signature status, signing identity, and code directory hashes.

- **`Signature=adhoc` & `flags=0x2(adhoc)`:**
  The binary is **ad-hoc signed** (signed locally with a pseudo-identity rather than an Apple Developer Certificate).
- Why? Apple Silicon requires *some* signature to execute without kernel rejection. Ad-hoc signing fulfills this minimal CPU architecture requirement without registering a paid Apple Developer account.
- **`TeamIdentifier=not set`:**
  There is no Apple Developer Team ID tied to this specimen.
- **`CDHash=90089ad92e2eb0688a8ea060a752e18128539253`:**
  The **Code Directory Hash (CDHash)** is one of the most critical IoCs in macOS threat hunting. It is a cryptographic hash of the binary's code directory pages.

macOS security subsystems (like Endpoint Security Framework, Endpoint Detection and Response tools, and XProtect) track binaries by CDHash rather than just the full file SHA-256.

> **Gatekeeper Impact:** Because the binary lacks a valid Developer ID signature and Apple Notarization ticket, if a user downloads this file via a web browser or email client, macOS applies the `com.apple.quarantine` extended attribute (`xattr`). Gatekeeper will immediately block execution with a warning dialog.
>
> To bypass this, campaigns like RustBucket rely on an initial first-stage dropper or AppleScript droplet that writes the `.pd` payload directly to disk via command-line utilities (like `curl` or shell scripts), intentionally omitting the quarantine attribute.

Gatekeeper bypass technique

Because this file lacks an Apple Developer ID signature and notarization ticket, downloading it through Safari, Chrome, or Mail sets the **com.apple.quarantine** extended attribute. Gatekeeper would block execution immediately.

To bypass Gatekeeper, the RustBucket campaign relied on a first-stage dropper or AppleScript droplet. The initial dropper writes **.pd** directly to disk through shell commands (such as **curl**), avoiding the quarantine flag entirely.

Static triage cheat sheet

When a new macOS specimen arrives for analysis, these commands provide basic architectural context within a minute:

```
# 1. identify file format and architecture slices
file sample

# 2. inspect universal binary offsets and slice sizes
otool -f sample

# 3. extract a standalone architecture slice
lipo sample -thin arm64 -output sample_arm64

# 4. view Mach-O header flags and capabilities
otool -h sample_arm64

# 5. extract memory segment layouts and the entry point
```

```
otool -l sample_arm64 | grep -E "(cmd LC_MAIN|entryoff|segname)"

# 6. list linked frameworks and dynamic libraries
otool -L sample_arm64

# 7. dump literal ASCII strings from the text segment
otool -v -s __TEXT __cstring sample_arm64

# 8. check code signing attributes and CDHash
codesign -dv --verbose=4 sample_arm64
```

Coming Up in Part 2: Disassembly & Dynamic Behavior

Now that we understand the basics of how a macOS binary is structured and runs, it's time to see what happens inside the code.

When you open pd_arm64 in a disassembler like Ghidra or IDA Pro, this address (0x1000026A4) is precisely where the decompilation flow starts.

In Part 2, we'll take a closer look at the RustBucket sample and follow its behavior step by step.

See you in Part 2!

Cybersecurity + Malware Analysis + Reverse Engineering + Macos +



Follow

Written by CHANDRA KANT BAURI

39 followers · 17 following

Malware Analyst Intern